Contributions on Schedule-Aware Bundle Routing

Projet de Programmation 2

DAGUE Bastien, AL-FARAJ Samy, JOUINI Yassine, BROCHARD Vincent

L3 Informatique - Faculté des Sciences - Université de Montpellier

2025 - 2026

Table des matières

Glossaire et Abréviations **2**

1 Introduction **3**
1.1 A-SABR . . . 3
1.2 Sujet . . . 3

2 Développements Logiciel : Conception, Modélisation, Implémentation **4**
2.1 DT-Chat . . . 4
2.1.1 DT-Chat Backend . . . 4
2.1.2 DT-Chat Frontend . . . 5
2.2 Pipeline . . . 5
2.3 Test A-SABR . . . 5
2.3.1 Module : Contact Manager . . . 5
2.3.1.1 Legacy : . . . 5
2.3.1.2 Segmentation : . . . 5
2.3.1.3 PrioritySegmentation : . . . 5
2.3.2 Module : Pathfinding . . . 5
2.3.2.1 Try_make_hop : . . . 5
2.3.2.2 Contact,Hybrid et Node parenting : . . . 5
2.4 BP-Socket . . . 6

3 Algorithmes et Structures de Données **7**

4 Analyse des résultats **8**

5 Gestion du Projet **9**

6 Bilan et Conclusions **10**

7 Annexe de codes **11**

Glossaire et Abréviations

BP Socket

Bundle Protocole Socket. Type spécifique de socket utilisée dans le cas des DTN [3]. 3

CP

Contact Plan. Fichier de configuration décrivant les fenêtres de communication planifiées entre nœuds : qui communique avec qui, à quelle heure, pendant combien de temps, avec quel débit et quel délai. Lu par la bibliothèque, il est transformé en Multigraph (Multigraph). 3

DTN

Delay-Tolerant Networking. Réseau tolérant aux délais et aux ruptures de connectivité, où les nœuds ne sont pas forcément connectés en permanence. Les données sont transmises de manière opportuniste via des contacts planifiés.. 3

Multigraph

Structure de données centrale du projet représentant le réseau. Contient les nœuds (senders/receivers) et les contacts. Permet plusieurs contacts entre une même paire de nœuds à des moments différents, d'où le préfixe "multi". 4

PRELOAD

Mécanisme du Dynamic Loader des systèmes UNIX permettant de charger un module dont les fonctions seront utilisées en priorités avant celles de la glibC. 3

SABR

Schedule-Aware Bundle Routing. Algorithme de routage pour réseaux DTN qui utilise des métriques de distance (temps d'arrivée, nombre de hops) pour trouver les meilleures routes entre nœuds. [1]. 3

1 Introduction

1.1 A-SABR

Notre projet de programmation porte sur la librairie ASABR, portée par Mr Olivier De Jonckère du LIRMM.

Le réseau de communication spacial repose sur le Delay-Tolerant Networking (DTN). Avec ce type de protocole, le routage s'effecture par des algorithmes de type Schedule-Aware Bundle Routing (SABR).

La librairie A-SABR implémente donc ce type d'algorithme en proposant une grande modularité et une bonne performance.[1] C'est également pour suivre ce choix de performance que celle-ci est programmé en Rust, language connue pour son compilateur stricte et sa fine gestion de la mémoire.

1.2 Sujet

L'intérêt pour ce sujet a été immédiat au sein de notre groupe, motivé par la perspective de contribuer à un projet Open Source d'envergure réelle. Ce cadre de travail nous a offert l'opportunité de nous initier au langage Rust, dont la montée en puissance dans l'industrie logicielle souligne l'importance actuelle. Pour plusieurs membres de l'équipe aspirant à se spécialiser en informatique système (bas niveau) ou en réseaux, ce projet constituait une application concrète et exigeante de nos ambitions professionnelles.

Le sujet présentait plusieurs point à aborder :

- Portage au no-std
- Divers tâche de DevOps (Tests, Pipeline)
- Création d'un nouvel onglet dans l'interface graphique DT-Chat
- Intégration dans le logiciel Hardy

Au final, tout à été un peu chamboulé, le no-std à été abbandonnée, l'intégration à Hardy jugée peut intéressante.

Nous avons surtout compris qu'ici on ne dépendait pas seulement de nous mais qu'il s'agissait d'un immense travail d'équipe. On commence une tâche, on la trouve accompli, on lance une Pull Request, on discute, on modifie, tous ça prend un certain temps et nécessite beaucoup de vas et viens.

Arrivé à la fin du projet voici les réels objectifs accompli :

Interface graphique DT-Chat : Développement d'un volet « Options » dans l'interface graphique DT-Chat : Ce module permet désormais de modifier dynamiquement, au cours de l'exécution, l'algorithme de routage utilisé ou le Contact Plan (CP) chargé.

Automatisation de test unitaires : Mise en place d'une pipeline CI/CD robuste pour assurer la pérennité de la bibliothèque. Ce travail a été complété par la mise en place de plusieurs tests unitaires couvrant les modules principaux du projet. La pipeline teste la compilation, l'execution des tests ainsi que la rédaction de la coverage sous forme d'HTML

Expérimentation Système (BP Socket) : Pendant le dernier mois du projet, nous avons travaillé sur Bundle Protocole Socket (BP Socket).

BP Socket telle qu'elle à été développée utilise un module kernel nécessitant les droit root pour être utilisée. L'objectif est de permettre l'usage de BP Socket sur des systèmes embarqués contraints où les privilèges « root » ne sont pas disponibles. L'expérimentation du mécanisme LD_PRELOAD (PRELOAD) vise à intercepter les appels système pour s'affranchir de la dépendance au module noyau actuel.

Ce dernier objectif est une total découverte de ce mécanisme offert par le Dynamic Loader et toute avancée même non complète intéresse Mr De Jonckère.

La compilation de la librairie et de l'interface graphique nécessitant divers paquet Linux, notre travail à été réalisé soit par WSL, soit directement depuis une distribution Linux.

2 Développements Logiciel : Conception, Modélisation, Implémentation

2.1 DT-Chat

DT-Chat est une application graphique codée en Rust permettant d'illustrer une communication DTN. Elle supporte UDP,TCP et BP Socket.

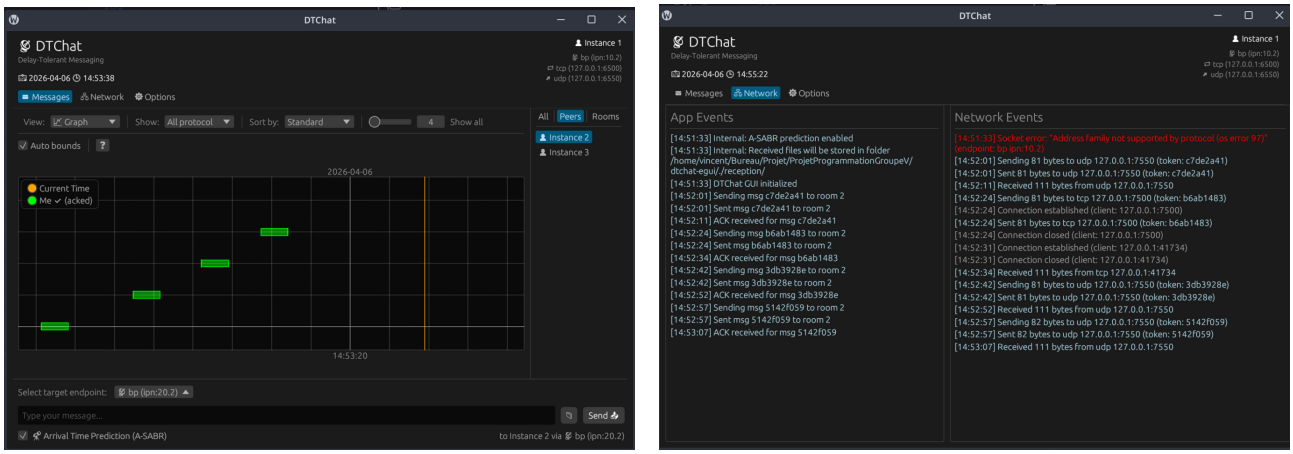

FIGURE 1 – Onglet Message (gauche) et Events (droite) d'un terminal DT-Chat

A gauche nous pouvons voir l'onglet principal, nous voyons un graphique des différents message envoyée, un rectangle vert signifie un message envoyée et reçu avec confirmation. En bas il y a la possibilité de choisir entre UDP,TCP et BP Socket. Egalement, il y a la possibilité de cocher la méthode A-SABR, ceci permet qu'a l'envoie d'un message l'application prédit et indique le moment où le message devrait être reçu par le recepteur.

A droite nous voyons l'onglet Event, celui ci recapitule de manière textuelle toute interaction de l'interface. A gauche le recapitulatif de l'interaction logicielle et à droite celle du reseau, on voit bien les différents message envoyée et reçu.

Sur cette exemple, il y a eu 3 envois du même terminal, 1 UDP, 1 TCP, 1 BP Socket. BP Socket dans le network Event est marqué comme UDP car il s'agit d'une variante d'UDP.

L'objectif sur cette interface graphique est de créer un nouvel onglet Options permettant de changer à la volée de CP et/ou d'algorithme. Cela nécessite donc un travail à la fois coté Backend et FrontEnd

Maintenant, détaillons le travail effectuée sur chacun des parties de cette interface.

2.1.1 Backend

7 (Cliquez sur le numéro pour accéder au code)

Premièrement, on créer une fonction update tiré de la fonction start de l'interface, celle-ci prend en argument un str et un chemin de fichier. Il faut aussi modifier la fonction de lancement de l'interface, qui ici se nomme try_init pour qu'elle prenne également un str et un chemin de fichier, on récupère également les valeurs de départ qu'on met dans l'appel de la fonction. try_init à aussi été modifie pour retourner le nombre de node et de contact trouvée dans le CP chargé.

La fonction update prend donc en entrée un str qui correspond au nom de l'algorithme utilisée, et un Path sous forme de string qui correspondra au chemin jusqu'au CP.

Elle va ensuite faire un match sur `"PredictionConfig::try_init(path.clone(), algo)"` qui retournera donc un Result, pour empêcher un crash total de l'interface au moindre soucis.

Si Ok alors on met à jour la configuration qu'on envoie au Frontent et on l'affiche dans l'App Event, si Err, on retourne l'erreur et on l'affiche dans l'App Event.

Etant basé sur A-SABR, il faut régulièrement faire de petite modif dûe à des refactorisation de fonction ou de l'ajout de Result remplaçant des Options dans la librairie.

Toute ces modifications sur la partie Backend ont fait l'objet de 2 Pull request (#6 et #7) acceptées pour 52 lignes de code ajoutée.

2.1.2 Frontend

2.2 Pipeline

Bastien : Ici detail point par point la pipeline l'évolution de ta PR, précise bien sur que ça à été PR etc etc

2.3 Test A-SABR

2.3.1 Contact Manager

2.3.1.1 Legacy :

Partie de Yassine et ou Samy

2.3.1.2 Segmentation :

Ici ta partie Yassine,

2.3.1.3 PrioritySegmentation :

Partie de Bastien

2.3.2 Pathfinding

Le rôle de ce module est de calculer, à partir d'un Bundle (Bundle) et d'un instant donnée, la meilleurs route vers une destination en tenant compte des contraites temporelles de chaque contact. Il expose un Trait, `PathFinding`, implémentée par trois variantes : Node, Contact et Hybride, détaillée ci-après, ainsi qu'un sous module `limiting_contact` qui permet de supprimer temporairement le contact le plus limitant d'une route pour forcer la découverte de routes alternatives.

Chaque variante est instanciée avec une métrique de distance, soit **Hop** qui minimise le nombre de saut, soit **SABR** qui minimise le temps d'arrivée.

2.3.2.1 Try_make_hop :

Ici ta partie Samy

2.3.2.2 Contact,Hybrid et Node parenting :

Contact,Hybride et Node sont 3 variantes de l'algorithme de Dijkstra qui sont utilisés pour le routage :

Node est le plus simple, il explore le Multigraph crée noeud par noeud, et ne maintient qu'une seule route optimal. Il trouve le chemin optimal selon la métrique choisie mais s'arrête dès que la destination est atteinte en mode Path, contrairement aux autres variantes qui continuent d'explorer.

Contact explore lui le Multigraph par contact plutôt que par noeud, il va utiliser une "Work Area", permettant d'explorer les contacts directement sans passer par la logique nœud par nœud, ce qui peut trouver des routes avec moins de saut dans certains cas.

Hybrid est la plus complète mais également la plus complexe, il va maintenir une liste de potentiel route optimal tous le long du parcours, puis, grâce au fonction "can_retain" et "must_prune", va décider laquelle garder.

Chacunes de ces variantes possédent les 3 mêmes macros :
- Path : Chemin simple vers une destination, s'arrête dès qu'elle est atteinte.
- PathExcl : Si l'on veut exclure un noeud précis.
- TreeExcl : Explore l'entièreté du Multigraph avec exclusion.

Les tests sonts simples, on créer plusieurs Multigraph auquel on applique chacunes des macros, on utilise le manager le plus simple possible pour ne pas rendre le test trop lourd, on applique en dernier la méthode Hop (le moins de saut) ou la méthode SABR (le plus rapide) pour avoir le plus de données de test possible.

Les résultats attendues sont le nombre de saut réalisés, et le temps d'arrivée. Pour les cas d'exclusion, on donne comme objectif par exemple le noeud 3 en excluant le noeud 2 et l'on doit avoir en retour un échec du au noeud exclus.

Dans un soucis de lisibilité et de maintienabilité du code, toute les fonctions de création d'objets (noeud,contact,bundle,multigraph) sont réunis dans un fichiers à part (test_helpers). De cette manière, seul les tests sont présents dans leur fichiers cibles.

Prenons un test en particulier et décryptons le : 7 (Cliquer sur le numéro pour être envoyé à l'annexe correspondant)

Nous avons donc une première fonction qui va créer le Multigraph auquel on va applique nos tests, le multigraph créer possèdents 5 noeuds qui ont un manager neutre (qui ne fait rien) pour faire au plus simple et de 5 contacts.

Exemple : le premier contact émet du noeud 0 au noeud 1, début du contact à 0 et fin à 10, débit de 1 unité/seconde et aucun délai de propagation.

Ensuite nous créeons un Bundle à destination du noeud 4, sans priorité et sans taille précise.

On contruit les deux chemin, un avec la méthode SABR, l'autre avec la méthode Hop, puis nous lui appliquons notre fonction de test.

— Hop : On attend 3 sauts en 50 unité de temps.
— SABR : On attend 4 sauts en 50 unité de temps.

Le fait que le nombre de saut diverge entre Hop et SABR permet une plus grand couverture pour detecter une casse de la librarie.

Cette partie à fait l'objet de 2 Pull Request de Vincent acceptée pour 629 lignes rajoutée correspondant aux tests dans les 3 fichiers + la création de test_helpers. (`#38` et `#44`)

2.4 BP-Socket

3 Algorithmes et Structures de Données

4 Analyse des résultats

5 Gestion du Projet

6 Bilan et Conclusions

7 Annexe de codes

2.1.1 (Cliquer sur le chapitre pour revenir au texte correspondant)

la fonction update est tiré de DTChat Backend "src/dtchat.rs" ligne 218
pred_res est tiré de DTChat Backend "src/config/mod.rs" ligne 83
try_init est tiré de DTChat Backend "src/prediction.rs" ligne 30

```
    pub fn update(&mut self, path:String, algo: &str){
        match PredictionConfig::try_init(path.clone(), algo){
            Ok(update_config) => {
                self.a_sabr = ASabrInitState::Enabled(update_config);
                self.notify_observers(ChatAppEvent::Info(format!("Update done with : {path} and  {algo}
            }
            Err(error) => {
                self.a_sabr = ASabrInitState::Error(error.to_string());
                self.notify_observers(ChatAppEvent::Info("Update failed".to_string()));
            }
        }
    }

pub fn try_init(cp_path: String, algo : &str) -> io::Result<Self> {
        let (nodes, contacts) = IONContactPlan::parse::<NoManagement, EVLManager>(&cp_path)?;

        let nodes_length = nodes.len();
        let contacts_length = contacts.len();

        let node_index_map: HashMap<String, NodeID> = nodes
            .iter()
            .enumerate()
            .map(|(index, node)| (node.get_node_name().to_string(), index as NodeID))
            .collect();

        let router_box = build_generic_router::<NoManagement, EVLManager>(
            algo,
            nodes,
            contacts,
            None,
        ).expect("Failed to build router");

        let router: Box<dyn Router<NoManagement, EVLManager> + Send + Sync> =
            unsafe { std::mem::transmute(router_box) };
        // in seconds
        let cp_start_time = DTChatTime::now().timestamp_millis() as f64 / 1000.0;

        Ok(PredictionConfig {
            ion_to_node_id: node_index_map,
            router,
            cp_start_time,
            nodes_length,
            contacts_length,
        })
    }


let pred_res = PredictionConfig::try_init(cp_path_unwrapped,"VolCgrHybridParenting");
```

2.3.2.2 (Cliquer sur le chapitre pour revenir au texte correspondant)

```rust
pub(crate) fn exemple_2_graph()
-> Result<Rc<RefCell<Multigraph<NoManagement, EVLManager>>>, ASABRError> {
    Ok(Rc::new(RefCell::new(Multigraph::new(ContactPlan::new(
        vec![
            make_node(0, "source", NoManagement {}),
            make_node(1, "from_C0", NoManagement {}),
            make_node(2, "from_C2_C1", NoManagement {}),
            make_node(3, "from_C3", NoManagement {}),
            make_node(4, "from_C4", NoManagement {}),
        ],
        vec![
            make_contact::<NoManagement>(0, 1, 0.0, 10.0, 1.0, 0.0),
            make_contact::<NoManagement>(0, 2, 25.0, 35.0, 1.0, 0.0),
            make_contact::<NoManagement>(1, 2, 10.0, 20.0, 1.0, 0.0),
            make_contact::<NoManagement>(2, 3, 20.0, 40.0, 1.0, 0.0),
            make_contact::<NoManagement>(3, 4, 50.0, 60.0, 1.0, 0.0),
        ],
        None,
    )?)?)))
}

fn test_exemple_2() -> Result<(), ASABRError> {
        let mg = exemple_2_graph()?;

        let mut algo_hop = NodeParentingPath::<NoManagement, EVLManager, Hop>::new(mg.clone());
        let mut algo_sabr = NodeParentingPath::<NoManagement, EVLManager, SABR>::new(mg.clone());

        let bundle = make_bundle(4, 0, 0.0, 1000.0);

        let res_hop = algo_hop
            .get_next(0.0, 0, &bundle, &[][..])
            .expect("Routing Failed !");

        assert_time_hop(&res_hop, 4, 50.0, 3, "Hop");

        let res_sabr = algo_sabr
            .get_next(0.0, 0, &bundle, &[][..])
            .expect("Routing Failed !");

        assert_time_hop(&res_sabr, 4, 50.0, 4, "SABR");

        Ok(())
    }
}
```

Références

[1] J. A. Fraire, O. De Jonckère, and S. Burleigh *Routing in the space internet : A contact graph routing tutorial.*, Journal of Network and Computer Applications, 174, 102884.

[2] O. De Jonckère, L. Ma, and J. A. Fraire. *A-SABR : The Adaptive Library for Schedule-Aware Bundle Routing.*, WiSEE 2025 - IEEE International Conference on Wireless for Space and Extreme Environments, Oct 2025, Halifax, Canada.

[3] Sylvain Pierrot, Olivier de Jonckère, Scott Burleigh, Charley Geoffroy. *Design and Implementation of a Native Socket Interface for the Bundle Protocol.* , WiSEE 2025 - 13th Annual IEEE International Conference on Wireless for Space and Extreme Environments, Oct 2025, Halifax, Canada.